



Predavanje 5

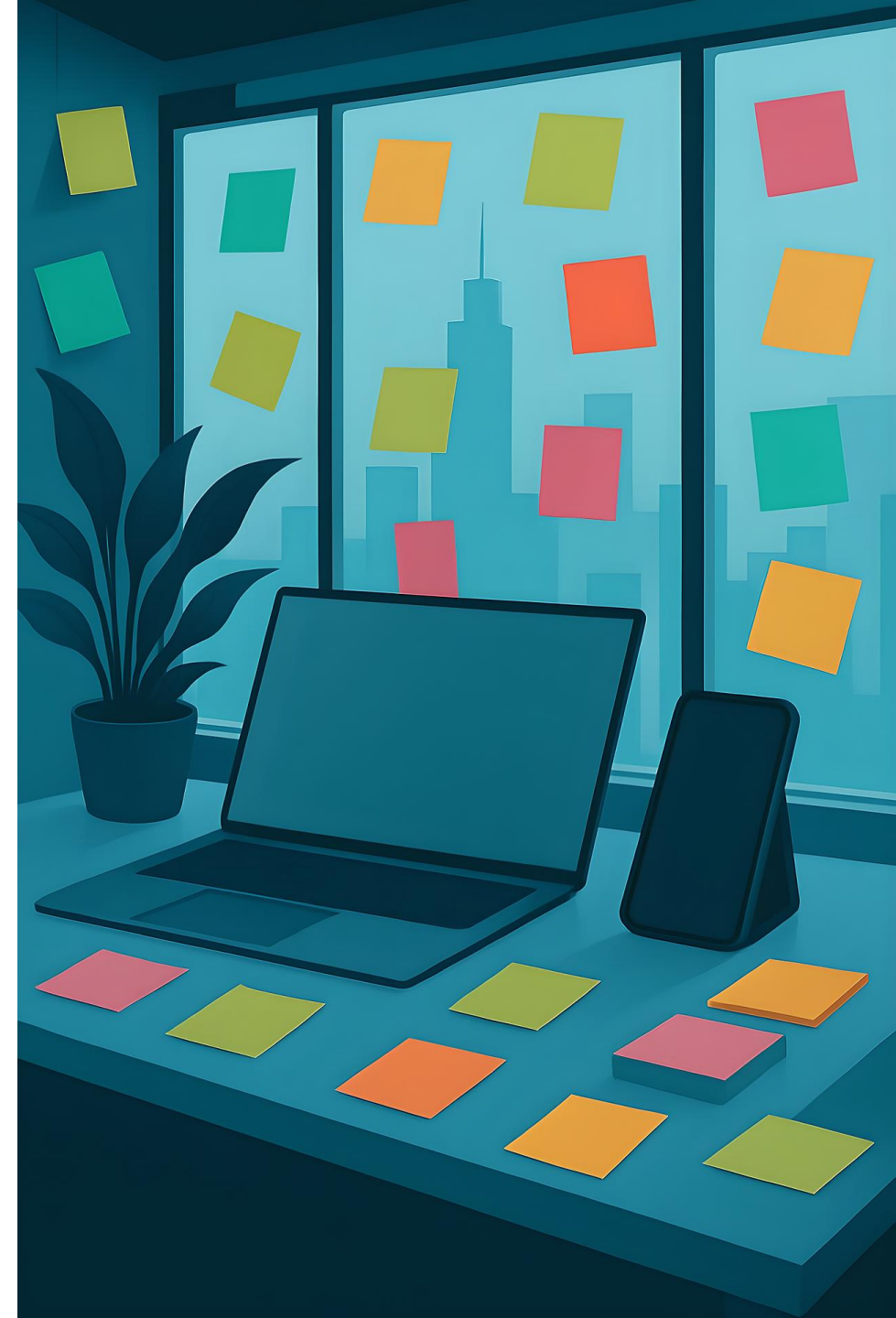
Programiranje za UNIX

Doc. dr. sc. Maja Braović

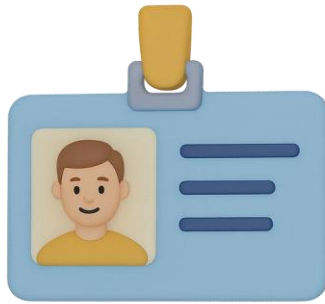
30. listopada 2025.

Sadržaj

- Uvod
- Rad sa datotekama
 - Deskriptor datoteke
 - STDIN preusmjeravanje
 - STDOUT preusmjeravanje
 - STDERR preusmjeravanje
 - C funkcije za rad sa datotekama
- Važni pojmovi
- Literatura



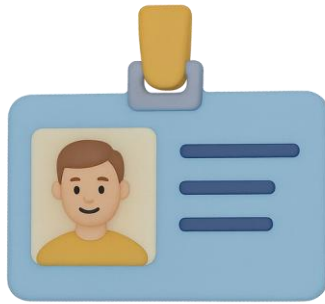
Deskriptor datoteke



Slika preuzeta iz [1].

- **Deskriptor ili opisnik datoteke** (eng. *file descriptor*) je nenegativni cijeli broj koji se koristi za komunikaciju sa otvorenom datotekom i koji ju jedinstveno identificira.
- Deskriptor datoteke se dodjeljuje programu koji traži otvaranje datoteke, a dodjeljuje ga jezgra operacijskog sustava.
- Program koristi deskriptor datoteke za daljnju komunikaciju s tom datotekom (npr. za čitanje ili pisanje podataka), te ne moraju znati gdje se ta datoteka fizički nalazi.
- Deskriptori datoteka su ključni u UNIX i UNIX-sličnim operacijskim sustavima jer služe za učinkovito upravljanje resursima i omogućavaju da više programa pristupa datotekama.

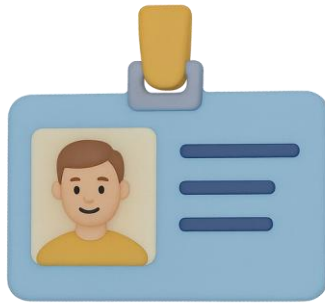
Deskriptor datoteke



Slika preuzeta iz [1].

- Proces dodjele deskriptora datoteke:
 1. Kad program pozove funkciju za otvaranje datoteke, jezgra prvi slobodni nenegativni broj(najniži mogući) koristi kao deskriptor datoteke.
 - Slobodni brojevi su **uvijek veći od 2**, jer su brojevi 0, 1 i 2 zauzeti.
 2. Jezgra zatim u svoju tablicu ubaci podatke o toj datoteci, uključujući pristupna prava, pokazivač na podatke datoteke, i stanje.
 3. Taj broj se vraća kao deskriptor datoteke, koji program koristi za sve buduće operacije nad tom datotekom, poput čitanja ili pisanja.
 4. Kad se datoteka zatvori, jezgra oslobađa deskriptor datoteke da bi ga mogla dodijeliti nekoj drugoj datoteci u budućnosti.

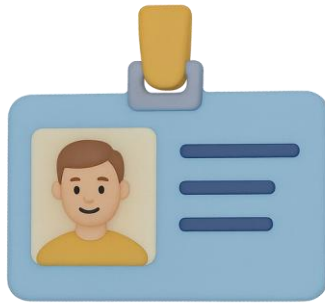
Deskriptor datoteke



Slika preuzeta iz [1].

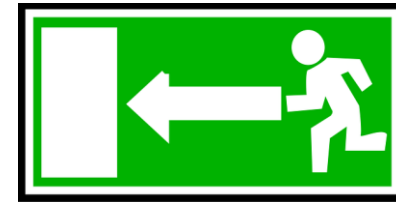
- Postoje standardni deskriptori datoteka koji su uvijek otvoreni za svaki proces u UNIX i UNIX-sličnim operacijskim sustavima:
 - **deskriptor 0** – standardni ulaz (**STDIN**) – koristi se za unos podataka u program, obično je to tipkovnica.
 - **deskriptor 1** – standardni izlaz (**STDOUT**) – koristi se za ispis podataka iz programa, obično na zaslon.
 - **deskriptor 2** – standardni izlaz za greške (**STDERR**) – koristi se za ispis poruka o greškama, također obično na zaslon.

Deskriptor datoteke



Slika preuzeta iz [1].

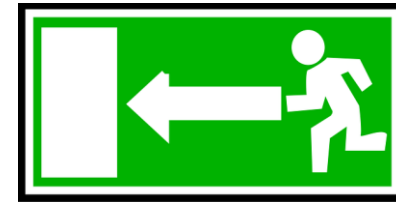
- Standardni datotečni deskriptori 0, 1 i 2 postali su konvencija još u UNIX sustavima 1970-ih godina. Tada su programeri morali ručno upravljati ovim deskriptorima.
- Kada su kreatori UNIX-a razvijali ovaj sustav, cilj im je bio što jednostavnije i učinkovitije omogućiti programima rad s ulazom i izlazom, pa su odlučili predstaviti sve uređaje i datoteke kao datoteke, dok su deskriptore definirali kao "**ključeve**" za pristup tim resursima.
- Podsjetimo se – u UNIX sustavima „**sve je datoteka**”.
- Datotečni deskriptor za STDERR omogućava programerima da uspješno odvoje standardni izlaz programa od **error** poruka, što je u današnje vrijeme od velike važnosti za automatizaciju i dijagnostiku.



Slika preuzeta iz [2].

STDIN preusmjeravanje

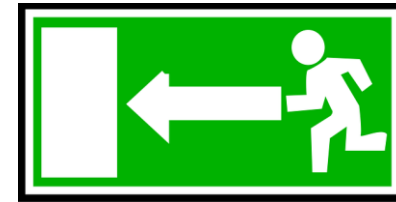
- Preusmjeravanje standardnog ulaza (**STDIN**) u UNIX i UNIX-sličnim operacijskim sustavima omogućuje da se podaci za ulaz umjesto sa tipkovnice uzimaju iz datoteke ili nekog drugog izvora.
- To se najčešće radi korištenjem operatora „<”.
- Primjer 1: **naredba < ulaz.txt**
 - Ova naredba neće čitati ulaz s tipkovnice, nego će kao ulaz koristiti sadržaj datoteke **ulaz.txt**.
 - Ovo je vrlo korisno kada želite automatizirati unos podataka ili koristiti unaprijed spremljene podatke kao ulaz naredbi.
- Primjer 2: **naredba 0< ulaz.txt**
 - Ovaj primjer je ekvivalentan prvom, budući da je „<” samo skraćeni način zapisa za „0<”.



Slika preuzeta iz [2].

STDOUT preusmjeravanje

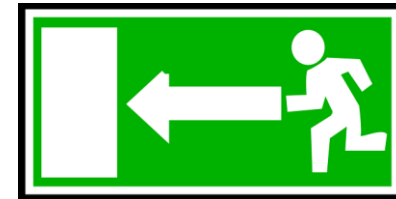
- Preusmjeravanje standardnog izlaza (**STDOUT**) u UNIX i UNIX-sličnim operacijskim sustavima omogućuje da se izlaz neke naredbe umjesto na ekran usmjeri u datoteku ili neki drugi uređaj.
- Najčešći načini STDOUT preusmjeravanja su:
 - „**Simbol >**” koristi se za preusmjeravanje standardnog izlaza u novu ili postojeću datoteku (koja se prepíše). Na primjer:
 - **ls -al > izlaz.txt**
 - Ova naredba šalje popis sadržaja direktorija u datoteku **izlaz.txt** umjesto na ekran.
 - „**Simbol >>**” koristi se za dodavanje izlaza na kraj postojeće datoteke bez brisanja njenog prethodnog sadržaja. Na primjer:
 - **ls -al >> izlaz.txt**
 - Ova naredba dodaje novi izlaz na kraj datoteke **izlaz.txt**.



Slika preuzeta iz [2].

STDOUT preusmjeravanje

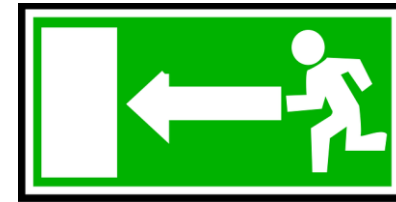
- Primjer 1: **naredba > izlaz.txt**
 - U ovom primjeru se ispis naredbe preusmjerava u datoteku **izlaz.txt**.
 - Ovdje nema potrebe za pisanjem „**1>**” (tj. „**file deskriptor 1 se preusmjerava u**”) jer ljuska (eng. *shell*) podrazumijeva da se radi o deskriptoru za standardni izlaz.
- Primjer 2: **naredba 1> izlaz.txt**
 - U ovom primjeru se ispis naredbe preusmjerava u datoteku **izlaz.txt**.
 - Ovaj primjer je jednak prvom primjeru, budući da je „**>**” samo duži način zapisa za „**1>**”.
- Primjer 3: **pwd > vjezba7.txt**
 - U ovom primjeru se ispis naredbe **pwd** preusmjerava u datoteku **vjezba7.txt**.



Slika preuzeta iz [2].

STDERR preusmjeravanje

- Preusmjeravanje standardnog izlaza za greške (**STDERR**) u UNIX i UNIX-sličnim operacijskim sustavima omogućuje da se izlaz koji signalizira greške neke naredbe umjesto na ekran usmjeri u datoteku ili neki drugi uređaj.
- To se najčešće radi korištenjem operatora „**2>**” za preusmjeravanje standardnog izlaza za greške u zasebnu datoteku.
- Primjer 1: **naredba 2> greske.txt**
 - Ova naredba preusmjerava sve poruke o greškama (stderr) u datoteku **greske.txt**, a standardni izlaz (stdout) ide i dalje na ekran.
- Primjer 2: **./vjeverica > izlaz.txt 2>&1**
 - Ova naredba preusmjerava standardni izlaz (stdout) u datoteku **izlaz.txt**, a zatim se standardni izlaz za greške (stderr) preusmjerava da ide na isto mjesto kao i standardni izlaz (što je sada datoteka **izlaz.txt**).



Slika preuzeta iz [2].

STDERR preusmjeravanje

- Primjer 3: **naredba** `> izlaz.txt 2>&1`
 - Ova naredba preusmjerava standardni izlaz (stdout) u datoteku **izlaz.txt**, a zatim se standardni izlaz za greške (stderr) preusmjerava da ide na isto mjesto kao i standardni izlaz (što je sada datoteka **izlaz.txt**).
 - Ako se simbol „&” nalazi nakon neke naredbe, onda se može koristiti za pokretanje procesa u pozadini.
 - U ovome slučaju, „`2>&1`” preusmjerava standardni izlaz za greške (stderr – **file descriptor 2**) na isto mjesto gdje ide standardni izlaz (stdout – **file descriptor 1**).
 - U ovoj je naredbi bitan zapis elemenata naredbe – najprije moramo specificirati gdje preusmjeravamo standardni izlaz („`> izlaz.txt`”), tako da bi pokazivač na standardni izlaz pokazivao na ispravnu lokaciju („`2>&1`”).

Funkcije za rad sa datotekama



Slika preuzeta iz [3].

- Postoji nekoliko standardnih funkcija za rad sa datotekama koje se koriste u programskom jeziku C:
 - `open()`
 - `creat()`
 - `close()`
 - `lseek()`
 - `read()`
 - `write()`
 - `umask()`

Funkcija **open()**



Slika preuzeta iz [3].

- Funkcija **open()** otvara datoteku čiju putanju specificira **pathname**, te vraća deskriptor datoteke koji predstavlja otvorenu datoteku.
- Ako dođe do pogreške prilikom otvaranja ili kreiranja datoteke, funkcija vraća -1. Moguće pogreške do kojih može doći su sljedeće:
 - Korisnik nema dopuštenje za pristup datoteci ili direktoriju.
 - Korisnik pokušava otvoriti postojeću datoteku koja ne postoji ili je netočno navedena putanja.
 - Korisnik pokušava kreirati datoteku koja već postoji.
 - Prekoračenje limita broja otvorenih datoteka u sustavu ili procesu.
 - Pokušaj korištenja nevažeće kombinacije zastavica, ili pogrešan način korištenja funkcije.

Funkcija **open()**



Slika preuzeta iz [3].

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
```

```
int open(const char *pathname, int oflag, /* mode_t mode */);
```

Funkcija **open()** – **sys/types.h**



Slika preuzeta iz [3].

```
#include <sys/types.h>
```

```
#include <sys/stat.h>
```

```
#include <fcntl.h>
```

```
int open(const char *pathname, int oflag, /* mode_t mode */);
```

Ova biblioteka služi za definiranje niza osnovnih sistemskih podatkovnih tipova (npr. **pid_t** za identifikatore procesa), koji se koriste u operacijskim sustavima i nizovima funkcija za pristup i upravljanje osnovnim resursima, poput datoteka, procesa i uređaja.

<https://pubs.opengroup.org/onlinepubs/009696699/basedefs/sys/types.h.html>

Funkcija **open()** – **sys/stat.h**



Slika preuzeta iz [3].

```
#include <sys/types.h>  
#include <sys/stat.h>  
#include <fcntl.h>
```

```
int open(const char *pathname, int oflag, /* mode_t mode */);
```

Ova biblioteka služi za definiranje strukture podataka i funkcija koje omogućavaju dohvat informacija o statusu (odatle ime „stat.h”) datoteka, kao što su veličina, prava pristupa, vrijeme zadnje izmjene i ostali atributi datoteke.

<https://pubs.opengroup.org/onlinepubs/7908799/xsh/sysstat.h.html>

Funkcija **open()** – **fcntl**



Slika preuzeta iz [3].

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

int open(const char *pathname, int oflag, /* mode_t mode */);
```

Ova biblioteka služi za kontrolu datotečnih deskriptora i upravljanje načinima otvaranja datoteka i njihovim zastavicama. Ime ove biblioteke je skraćenica za „file control”.

<https://pubs.opengroup.org/onlinepubs/009604499/basedefs/fcntl.h.html>

Funkcija **open()** – **pathname**



Slika preuzeta iz [3].

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

int open(const char *pathname, int oflag, /* mode_t mode */);
```

Ovaj parametar funkcije **open()** predstavlja putanju do datoteke.

Funkcija **open()** – **oflag**



Slika preuzeta iz [3].

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

int open(const char *pathname, int oflag, /* mode_t mode */);
```

Ovaj parametar funkcije predstavlja skup zastavica (eng. *flags*) koje specificiraju način na koji će se datoteka otvoriti ili kako će se upravljati s njom. Ime ovoga parametra kombinira prvo slovo riječi „output” te riječ „flag”.

Funkcija **open()** – **oflag**



Slika preuzeta iz [3].

- Vrijednost parametra **oflag** se sastoji od kombinacije bitovnih konstanti definiranih u biblioteci **<fcntl.h>**, kao što su:
 - **O_RDONLY** (otvaranje samo za čitanje)
 - **O_WRONLY** (otvaranje samo za pisanje)
 - **O_RDWR** (otvaranje za čitanje i pisanje)
 - **O_CREAT** (kreiranje datoteke ako ne postoji)
 - **O_APPEND** (dopisivanje na kraj datoteke)
 - **O_TRUNC** (skraćivanje datoteke na nulu)
 - **O_NONBLOCK** (neblokirajući način rada)
 - **O_EXCL** (skraćivanje datoteke na nulu)
 - **O_SYNC** (skraćivanje datoteke na nulu)

Funkcija **open()** – **oflag**



Slika preuzeta iz [3].

- Vrijednost parametra **oflag** se sastoji od kombinacije bitovnih konstanti definiranih u biblioteci **<fcntl.h>**, kao što su:

- **O_RDONLY** (otvaranje samo za čitanje)
- **O_WRONLY** (otvaranje samo za pisanje)
- **O_RDWR** (otvaranje za čitanje i pisanje)

- **O_CREAT** (kreiranje datoteke ako ne postoji)
- **O_APPEND** (dopisivanje na kraj datoteke)
- **O_TRUNC** (skraćivanje datoteke na nulu)
- **O_NONBLOCK** (neblokirajući način rada)
- **O_EXCL** (skraćivanje datoteke na nulu)
- **O_SYNC** (skraćivanje datoteke na nulu)

Parametar **oflag** mora sadržavati jednu od ove 3 konstante, te samo jedna od njih smije biti navedena u pozivu funkcije **open()**.

Funkcija **open()** – **oflag**



Slika preuzeta iz [3].

- Vrijednost parametra **oflag** se sastoji od kombinacije bitovnih konstanti definiranih u biblioteci **<fcntl.h>**, kao što su:

- **O_RDONLY** (otvaranje samo za čitanje)
- **O_WRONLY** (otvaranje samo za pisanje)
- **O_RDWR** (otvaranje za čitanje i pisanje)

- **O_CREAT** (kreiranje datoteke ako ne postoji)
- **O_APPEND** (dopisivanje na kraj datoteke)
- **O_TRUNC** (skraćivanje datoteke na nulu)
- **O_NONBLOCK** (neblokirajući način rada)
- **O_EXCL** (skraćivanje datoteke na nulu)
- **O_SYNC** (skraćivanje datoteke na nulu)

Parametar **oflag** mora sadržavati jednu od ove 3 konstante, te samo jedna od njih smije biti navedena u pozivu funkcije **open()**.

Ove konstante su opcionalne.

Funkcija **open()** – **oflag**



Slika preuzeta iz [3].

- Vrijednost parametra **oflag** – **O_APPEND**:
 - Pri svakom pisanju podataka sadržaj se dodaje na kraj datoteke.
- Vrijednost parametra **oflag** – **O_CREAT**:
 - Kreira novu datoteku ako ne postoji.
 - Ova opcija zahtijeva treći argument funkcije, koji specificira ovlasti nad datotekom.
- Vrijednost parametra **oflag** – **O_EXCL**:
 - Ako je naveden O_CREAT, a datoteka već postoji, funkcija vraća grešku.
 - Testiranje i kreiranje datoteke izvršava se kao **atomska** operacija, tj. operacija koja ne može biti prekinuta ni djelomično izvršena.

Funkcija **open()** – **oflag**



Slika preuzeta iz [3].

- Vrijednost parametra **oflag** – **O_TRUNC**:
 - Ako je datoteka otvorena za pisanje ili za pisanje i čitanje, njen sadržaj se briše, a dužina datoteke postaje 0.
- Vrijednost parametra **oflag** – **O_SYNC**:
 - Funkcija čeka da se svaka **write** operacija završi na fizičkom nivou prije nego što nastavi.
- Vrijednost parametra **oflag** – **O_NONBLOCK**:
 - Neblokirajući način rada znači da operacije čitanja ili pisanja koje se izvode na određenoj datoteci ili socketu neće blokirati (čekati) proces ako podaci trenutno nisu dostupni.
 - Umjesto toga, funkcija će odmah vratiti kontrolu programu, često uz informaciju da podaci nisu spremni, što omogućava programu da nastavi sa drugim zadacima bez čekanja.

Funkcija `open()` – `oflag`



Slika preuzeta iz [3].

- Parametar `oflag` je kombinacija konstanti prikazanih na prethodnim slajdovima.
- Te su konstante spojene bitovnim operatorom **OR (|)**, što omogućava precizno specificiranje željenog režima otvaranja datoteke i drugih svojstava.
- Primjer:
 - `int fd = open("datoteka.txt", O_WRONLY | O_CREAT | O_TRUNC);`
 - `O_WRONLY` postavlja datoteku u režim za pisanje.
 - `O_CREAT` znači da će se datoteka kreirati ako ne postoji.
 - `O_TRUNC` znači da će se postojeća datoteka skratiti na nulu prilikom otvaranja.

Funkcija **open()** – **mode**



Slika preuzeta iz [3].

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

int open(const char *pathname, int oflag, /* mode_t mode */);
```

Ovaj parametar predstavlja dozvole (ovlasti) koje će imati nova datoteka ukoliko se kreira (kada se koristi opcija O_CREAT), te definira prava pristupa datoteci (poput prava za čitanje, pisanje i izvršavanje).

Funkcija `open()` – `mode`



Slika preuzeta iz [3].

- Tip podataka `mode_t` u programskom jeziku C je cijeli broj (eng. *integer*) koji se koristi za predstavljanje prava pristupa datoteci, odnosno ovlasti nad datotekom.
- On definira skup bitova koji označavaju različite dozvole, kao što su dozvole za čitanje, pisanje i izvršavanje, koje se tada dodjeljuju novokreiranoj datoteci prilikom njena kreiranja.
- Parametar `mode` može poprimiti sljedeće vrijednosti:
 - `S_IRUSR` (dozvola za čitanje datoteke vlasniku, tj. useru)
 - `S_IWUSR` (dozvola za pisanje u datoteku vlasniku)
 - `S_IRGRP` (dozvola za čitanje datoteke članovima grupe kojoj datoteka pripada)
 - `S_IROTH` (dozvola za čitanje datoteke ostalim korisnicima)
- U gornjim vrijednostima „s” je skraćenica za „status” ili „set”, a „i” za „index”.

Funkcija **open()**



Slika preuzeta iz [3].

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main() {
    int fd = open("moja_datoteka.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1) {
        perror("open");
        return 1;
    }

    if (close(fd) == -1) {
        perror("close");
        return 1;
    }

    return 0;
}
```

Funkcija **creat()**



Slika preuzeta iz [3].

```
#include <sys/types.h>
```

```
#include <sys/stat.h>
```

```
#include <fcntl.h>
```

```
int creat(const char *pathname, mode_t mode);
```

Funkcija **creat()**



Slika preuzeta iz [3].

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

int creat(const char *pathname, mode_t mode);
```

Ova funkcija je ekvivalentna sa:

```
open (pathname, O_WRONLY | O_CREAT | O_TRUNC, mode);
```

Funkcija **creat()**



Slika preuzeta iz [3].

- Funkcija **creat()** se koristi za kreiranje nove datoteke ili skraćivanje postojeće na nulu, te vraća deskriptor za tu datoteku.
- Ova funkcija može otvoriti datoteku samo za pisanje.
- Datoteci se dodjeljuju prava pristupa zadana s **mode**, slično kao kod funkcije **open()**.
- Primjer:
 - **int fd = creat("moja_datoteka.txt", 0644);**
 - Ovom naredbom se kreira nova datoteka `moja_datoteka.txt`, te se prava pristupa postavljaju na 0644 (oktalni oblik - čitanje i pisanje vlasniku, samo čitanje ostalima).

Funkcija **creat()**



Slika preuzeta iz [3].

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main() {
    int fd = creat("moja_datoteka.txt", 0644);
    if (fd == -1) {
        perror("creat");
        return 1;
    }
    // Zapisivanje u datoteku
    write(fd, "Pozdrav!\n", 9);
    close(fd);
    return 0;
}
```


Funkcija **close()**



Slika preuzeta iz [3].

```
#include <unistd.h>
```

```
int close(int filedes);
```

- Biblioteka **unistd.h** definira sistemske funkcije za rad sa datotekama i procesima.
- Parametar **filedes** koji prima funkcija **close()** je deskriptor datoteke koju jezgra (kernel) treba zatvoriti.
- Kad proces (pokrenuti program) završi sa radom, sve otvorene file deskriptore zatvara kernel.
- Funkcija **close()** vraća 0 ako je sve u redu, te -1 u slučaju greške.

Funkcija **lseek()**



Slika preuzeta iz [3].

```
#include <sys/types.h>
#include <unistd.h>

off_t lseek(int filedес, off_t offset, int whence);
```

Funkcija **lseek()**



Slika preuzeta iz [3].

```
#include <sys/types.h>
#include <unistd.h>

off_t lseek(int filedes, off_t offset, int whence);
```

Ovo je file deskriptor otvorene datoteke (dobiven npr. iz funkcije **open()**).

Funkcija **lseek()**



Slika preuzeta iz [3].

```
#include <sys/types.h>
#include <unistd.h>

off_t lseek(int filedes, off_t offset, int whence);
```

Ovo je pomak u bajtovima koji će se primijeniti na položaj datoteke.

Funkcija `lseek()` – `offset`



Slika preuzeta iz [3].

- Tip podatka `off_t` je definirani cijelobrojni tip koji se koristi za predstavljanje položaja (offseta) unutar datoteke ili veličine datoteke u bajtovima.
- Obično je i dovoljno velik da može adresirati velike datoteke, što omogućuje rad s datotekama čiji je sadržaj veći od 2 GB, ovisno o platformi i implementaciji.
- Obično se koristi za precizno pozicioniranje i mjerenje unutar datoteka pri radu na niskoj razini.
- Standardni tipovi poput `int` ili `long` možda nisu dovoljni za veličine datoteka na modernim datotečnim sustavima, a `off_t` osigurava kompatibilnost s POSIX standardima i podršku za velike datoteke na promjenjivim platformama.

Funkcija **lseek()**



Slika preuzeta iz [3].

```
#include <sys/types.h>
#include <unistd.h>

off_t lseek(int filedес, off_t offset, int whence);
```

Ovo je način interpretacije **offset** parametra. Moguće vrijednosti su:

- **SEEK_SET** – offset u odnosu na početak datoteke.
- **SEEK_CUR** – offset u odnosu na trenutnu poziciju.
- **SEEK_END** – offset u odnosu na kraj datoteke.

Funkcija **lseek()**



Slika preuzeta iz [3].

- Funkcija **lseek()** služi za eksplicitno postavljanje i pomicanje file offset-a (trenutne pozicije čitanja ili pisanja) u otvorenoj datoteci.
- Funkcija vraća novi položaj (eng. *file offset*) u datoteci, -te 1 u slučaju greške.
- Kernel za svaku otvorenu datoteku prati trenutačni offset koji određuje odakle će se sljedeće čitanje ili pisanje dogoditi.
- Pomicanjem offseta na poziciju izvan kraja datoteke i pisanjem podataka, datoteka se može proširiti, a prazni dijelovi između prethodnog kraja i nove pozicije popunjavaju se nulama (ovisno o datotečnom sustavu).

Funkcija **lseek()**



Slika preuzeta iz [3].

- Slovo „l” u nazivu funkcije `lseek` dolazi od engleske riječi „long” i označava da funkcija radi sa vrijednostima tipa `off_t`, koje su (upravom zbog potrebe rada s velikim datotekama) često veće od standardnog `int` tipa.
- Drugim riječima, **`lseek()`** je funkcija za pomicanje (seek) pomoću long offseta (dugog pomaka) unutar datoteke.
- Budući da funkcija **`lseek()`** vraća novi file offset, pomicanjem offseta za 0 byteova od trenutne pozicije možemo odrediti trenutni file offset:
 - **`off_t currpos;`**
 - **`currpos = lseek(fd, 0, SEEK_CUR);`**

Funkcija `lseek()` – primjer 1



Slika preuzeta iz [3].

```
#include <sys/types.h>
...

int main() {
    char buf1[] = "First line of text";
    char buf2[] = "Second line of text";
    int fd;

    fd = creat("file.hole", (mode_t)0644);
    if (fd == -1)
        perror("creat");

    if (write(fd, buf1, strlen(buf1)+1) != strlen(buf1)+1)
        perror("write buf1");
    if (lseek(fd, 20, SEEK_CUR) == -1)
        perror("lseek");
    if (write(fd, buf2, strlen(buf2)+1) != strlen(buf2)+1)
        perror("write buf2");

    close(fd);
    exit(0);
}
```

Funkcija `lseek()` – primjer 1



Slika preuzeta iz [3].

```
#include <sys/types.h>
...

int main() {
    char buf1[] = "First line of text";
    char buf2[] = "Second line of text";
    int fd;

    fd = creat("file.hole", (mode_t)0644);
    if (fd == -1)
        perror("creat");

    if (write(fd, buf1, strlen(buf1)+1) != strlen(buf1)+1)
        perror("write buf1");
    if (lseek(fd, 20, SEEK_CUR) == -1)
        perror("lseek");
    if (write(fd, buf2, strlen(buf2)+1) != strlen(buf2)+1)
        perror("write buf2");

    close(fd);
    exit(0);
}
```

Kreira se datoteka "file.hole" s pravima 0644 (čitanje i pisanje za vlasnika, čitanje za grupu i ostale).

Funkcija `lseek()` – primjer 1



Slika preuzeta iz [3].

```
#include <sys/types.h>
...

int main() {
    char buf1[] = "First line of text";
    char buf2[] = "Second line of text";
    int fd;

    fd = creat("file.hole", (mode_t)0644);
    if (fd == -1)
        perror("creat");

    if (write(fd, buf1, strlen(buf1)+1) != strlen(buf1)+1)
        perror("write buf1");
    if (lseek(fd, 20, SEEK_CUR) == -1)
        perror("lseek");
    if (write(fd, buf2, strlen(buf2)+1) != strlen(buf2)+1)
        perror("write buf2");

    close(fd);
    exit(0);
}
```

Piše sadržaj buf1 ("First line of text")
u datoteku, uključujući i null
terminator (+1).

Funkcija `lseek()` – primjer 1



Slika preuzeta iz [3].

```
#include <sys/types.h>
...

int main() {
    char buf1[] = "First line of text";
    char buf2[] = "Second line of text";
    int fd;

    fd = creat("file.hole", (mode_t)0644);
    if (fd == -1)
        perror("creat");

    if (write(fd, buf1, strlen(buf1)+1) != strlen(buf1)+1)
        perror("write buf1");
    if (lseek(fd, 20, SEEK_CUR) == -1)
        perror("lseek");
    if (write(fd, buf2, strlen(buf2)+1) != strlen(buf2)+1)
        perror("write buf2");

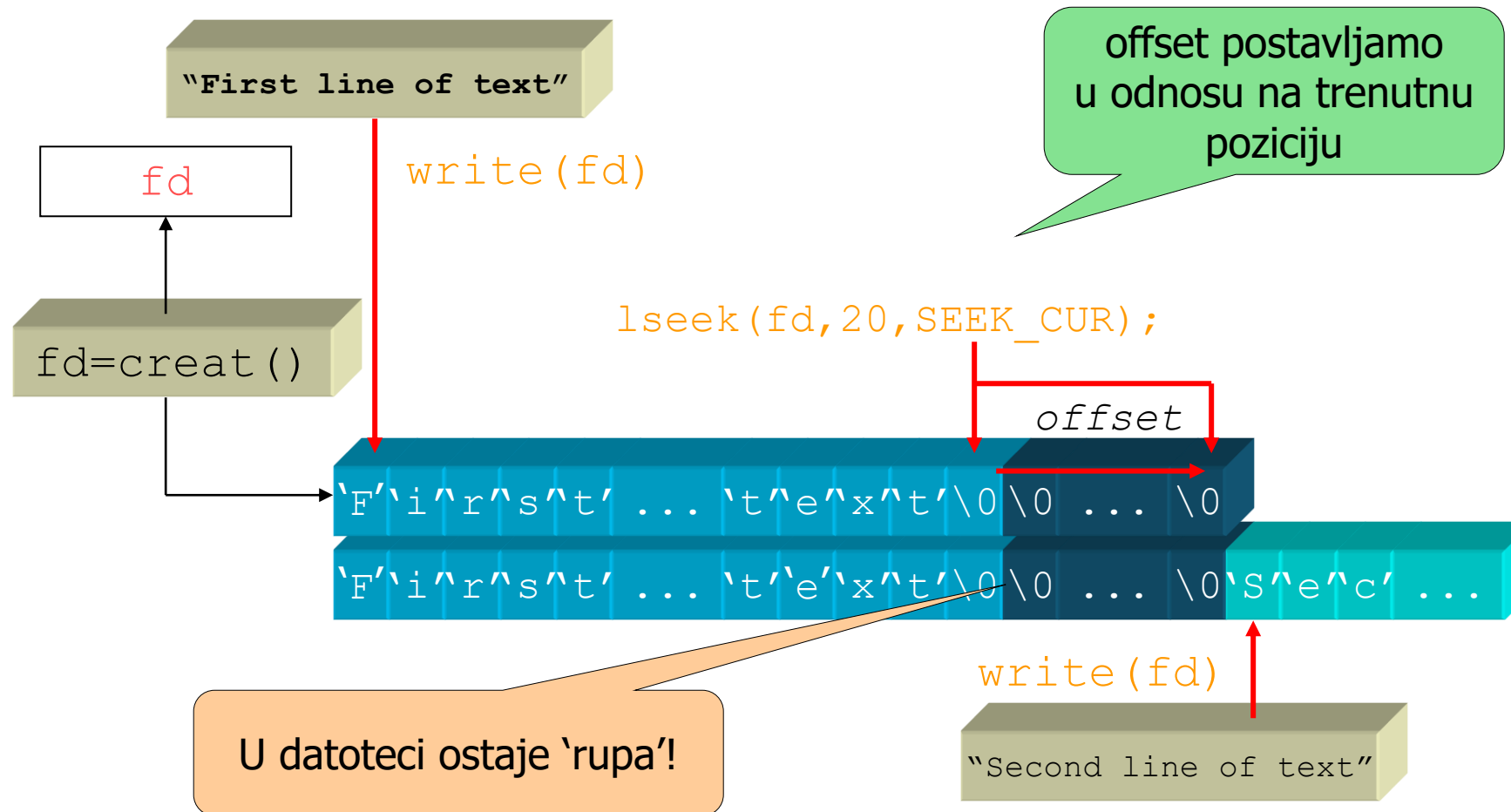
    close(fd);
    exit(0);
}
```

Pomiče trenutnu poziciju podataka za 20 bajtova unaprijed u odnosu na trenutnu poziciju (`SEEK_CUR`).

Funkcija `lseek()` – primjer 1



Slika preuzeta iz [3].



Funkcija `lseek()` – primjer 2



Slika preuzeta iz [3].

```
#include <sys/types.h>
...

int main() {
    char buf1[] = "First line of text";
    char buf2[] = "Second line of text";
    int fd;

    fd = open("file.strip", O_WRONLY | O_CREAT | O_TRUNC, (mode_t)0644);
    if (fd == -1)
        perror("open");

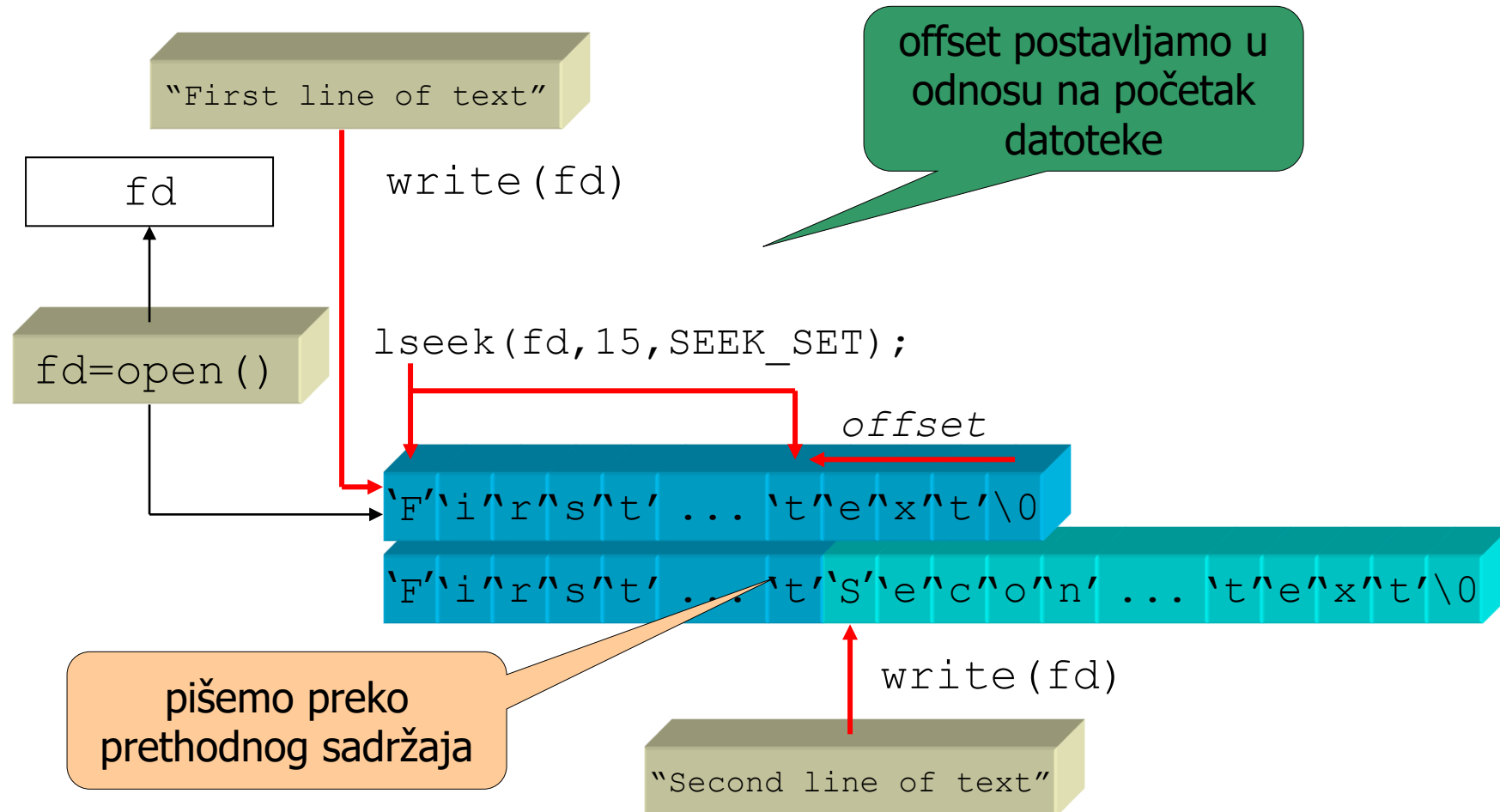
    if (write(fd, buf1, strlen(buf1)+1) != strlen(buf1)+1)
        perror("write buf1");
    if (lseek(fd, 15, SEEK_SET) == -1)
        perror("lseek");
    if (write(fd, buf2, strlen(buf2)+1) != strlen(buf2)+1)
        perror("write buf2");

    close(fd);
    exit(0);
}
```

Funkcija `lseek()` – primjer 2



Slika preuzeta iz [3].



Funkcija **read()**



Slika preuzeta iz [3].

```
#include <unistd.h>
```

```
ssize_t read(int filedes, void *buff, size_t nbytes);
```


Funkcija **read()**



Slika preuzeta iz [3].

```
#include <unistd.h>
```

```
ssize_t read(int filedes, void *buff, size_t nbytes);
```

Funkcija **read()** služi za čitanje podataka iz datoteke ili nekog drugog ulaznog toka i upisivanje tih podataka u memorijski prostor.

Funkcija vraća broj pročitanih byteova, 0 za kraj datoteke, te -1 u slučaju greške.

Funkcija **read()**



Slika preuzeta iz [3].

```
#include <unistd.h>

ssize_t read(int filedes, void *buff, size_t nbytes);
```

Ovo je file deskriptor datoteke.

Funkcija **read()**



Slika preuzeta iz [3].

```
#include <unistd.h>
```

```
ssize_t read(int filedes, void *buff, size_t nbytes);
```

Ovaj parametar predstavlja pokazivač na blok memorije (bajtove) u koji će se upisati podaci pročitani iz datoteke.

Funkcija **read()**



Slika preuzeta iz [3].

```
#include <unistd.h>

ssize_t read(int filedes, void *buff, size_t nbytes);
```

Ovaj parametar predstavlja maksimalan broj bajtova koji će funkcija pokušati pročitati iz datoteke u buffer **buff**.

Funkcija **read()**



Slika preuzeta iz [3].

```
#include <unistd.h>
```

```
ssize_t read(int filedes, void *buff, size_t nbytes);
```

Ovo je nepotpisani cjelobrojni tip (eng. *unsigned integer type*) podataka koji se koristi za predstavljanje veličine objekata u memoriji ili za broj elemenata.

Nepotpisani tipovi podataka nemaju negativnih vrijednosti.

Funkcija **read()**



Slika preuzeta iz [3].

```
#include <unistd.h>
```

```
ssize_t read(int filedes, void *buff, size_t nbytes);
```

Ovo je tip podataka koji je sličan tipu **size_t**, ali je potpisan (eng. *signed*). To znači da može prikazivati i negativne vrijednosti, što je korisno za funkcije i sistemske pozive koji mogu vratiti i negativne vrijednosti kao oznaku pogreške.

Funkcija `read()` – `ssize_t`



Slika preuzeta iz [3].

- Tip podataka `ssize_t` je standardiziran POSIX tip posebno dizajniran za izražavanje količine pročitanih ili zapisanih bajtova, uključujući mogućnost vraćanja negativnih vrijednosti za označavanje pogrešaka (-1).
- Korištenjem `ssize_t` jasno se daje do znanja da je riječ o mjeri veličine ili broja bajtova koja može imati i negativnu vrijednost.
- Razlika u značenju i kontekstu sa `int` tipom podataka:
 - `int` se koristi za opće cijele brojeve s potencijalnim negativnim vrijednostima, ali nema semantičku vezu s veličinama ili količinama podataka
 - `ssize_t` jasno označava da se radi o veličini (eng. *size*) koja može biti negativna u slučaju pogreške.

Funkcija **write()**



Slika preuzeta iz [3].

```
#include <unistd.h>
```

```
ssize_t write(int filedes, void *buff, size_t nbytes);
```


Funkcija **write()**



Slika preuzeta iz [3].

```
#include <unistd.h>
```

```
ssize_t write(int filedes, void *buff, size_t nbytes);
```

Funkcija **write()** služi za zapisivanje podataka iz memorije u datoteku ili na neki drugi izlazni tok (npr. uređaj, socket).

Funkcija **write()**



Slika preuzeta iz [3].

- Funkcija **write()** vraća broj upisanih byteova, te -1 u slučaju greške.
- Pisanje počinje na trenutnom offsetu. Prije povratka iz funkcije offset se povećava za broj upisanih byteova.
- Ukoliko je kod otvaranja datoteke uključena **O_APPEND** opcija, offset se prije svakog pisanja postavlja na kraj datoteke.

Funkcija **umask()**



Slika preuzeta iz [3].

```
#include <sys/types.h>
#include <sys/stat.h>

mode_t umask(mode_t cmask);
```

Funkcija **umask()** vraća prethodnu vrijednost maske koju je proces imao, prikazane u oktalnom obliku.

Funkcija **umask()**



Slika preuzeta iz [3].

- Funkcija **umask()** postavlja **masku** za prava pristupa novih datoteka i direktorija koje proces kreira.
- Ta **maska određuje koja će prava biti isključena (maskirana)** kad se datoteka ili direktorij stvaraju.
- Kad se kreira nova datoteka ili direktorij, operacijski sustav najprije pokušava dati prava prema zadanim vrijednostima (npr. 666 za datoteke ili 777 za direktorije), a zatim se ta prava "presijeku" pomoću **umask()** maske, koja onemogućava određene dozvole.
- Funkcija **umask()** pomaže korisnicima i programerima da kontroliraju koje će dozvole nove datoteke imati. Tako se sprječava da datoteke dobiju prevelika, nepoželjna prava pristupa.

VLASNIK

GRUPA

OSTALI

-	r	w	x	r	w	-	r	w	-
---	---	---	---	---	---	---	---	---	---

0	4	2	1	4	2	0	4	2	0
---	---	---	---	---	---	---	---	---	---

$$4 + 2 + 1 = 7$$

$$4 + 2 + 0 = 6$$

$$4 + 2 + 0 = 6$$

chmod 0766 datoteka.txt

Funkcija **umask()**



Slika preuzeta iz [3].

- Primjer:
 - Pretpostavimo da je zadani način za stvaranje datoteke **666 (rw-rw-rw-)**, što znači da svi imaju pravo čitanja i pisanja.
 - Ako je **umask** postavljen na **022**, prava koja se isključuju su:
 - **0 za vlasnika** – znači da nijedno pravo nije isključeno, dakle vlasnik zadržava sva prava koja su trebala biti dodijeljena
 - **2 za grupu** – isključuje se pravo pisanja
 - **2 za ostale** – isključuje se pravo pisanja.
 - Zato će konačna prava na datoteci biti **644 (rw-r--r--)**, što znači da vlasnik može čitati i pisati, a grupa i ostali samo čitati.

Funkcija **umask()**



Slika preuzeta iz [3].

- Primjer:

KORISNICI	666 (ZADANO)	- 022 (UMASK)	= KONAČNA PRAVA
Vlasnik	rw- (6)	/	rw- (6)
Grupa	rw- (6)	-w- (2)	r-- (4)
Ostali	rw- (6)	-w- (2)	r-- (4)

Funkcija **umask()**



Slika preuzeta iz [3].

- Standardna (eng. *default*) vrijednost maske **umask** na većini UNIX/Linux sustava je 022.
- Vrijednost 022 znači da će nove datoteke imati prava 644 (666 minus 022), a direktoriji 755 (777 minus 022). To osigurava da vlasnik datoteke ima sva prava (čitati i pisati), dok grupa i ostali imaju pravo čitanja i izvršavanja/dodavanja prava, ali ne i pisanja.
- Ova vrijednost je najčešće postavljena defaultno na sistemima zbog sigurnosnih razloga, jer ograničava pristup drugim korisnicima.
- Funkcija **umask()** se primjenjuje globalno za proces i utječe na sve nove datoteke koje se kreiraju u okviru tog procesa.
- Ukratko, **umask()** je standardni mehanizam u UNIX i UNIX-sličnim sustavima sustavima za upravljanje i ograničenje prava pristupa novih datoteka i direktorija.

Funkcija **umask()**



Slika preuzeta iz [3].

```
#include <sys/types.h>
#include <sys/stat.h>

mode_t umask(mode_t cmask);
```

Argument **cmask** funkcije **umask()** se formira disjunkcijom (logičko ili - OR) konstanti za prava pristupa datotekama (S_IRUSR, S_IWUSR, S_IXUSR, ...).

Ovi simboli (npr. S_IRUSR) se koriste u programiranju na UNIX i UNIX sličnim sustavima kao simbolički nazivi za određena prava pristupa koja se postavljaju na datoteke i direktorije.

Funkcija **umask()**



Slika preuzeta iz [3].

st_mode mask	Značenje
S_IRUSR S_IWUSR S_IXUSR	user-read user-write user-execute
S_IRGRP S_IWGRP S_IXGRP	group-read group-write group-execute
S_IROTH S_IWOTH S_IXOTH	other-read other-write other-execute

Funkcija `umask()` – `perm_mask.c`



Slika preuzeta iz [3].

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>

int main() {
    umask(0);
    if (creat("test_foo", S_IRUSR | S_IWUSR | \
        S_IRGRP | S_IWGRP | S_IROTH) < 0)
        perror("creat error for test_foo");

    umask(S_IRGRP | S_IWGRP | S_IROTH | S_IWOTH);
    if (creat("test_bar", S_IRUSR | S_IWUSR | \
        S_IRGRP | S_IWGRP | S_IROTH) < 0)
        perror("creat error for test_bar");

    exit(0);
}
```

Funkcija `umask()` – `perm_mask.c`



Slika preuzeta iz [3].

- Program `perm_mask.c` kreira datoteke sa istim navedenim pravima pristupa, ali sa različitom maskom za mod kreiranja datoteke.
- Primjer korištenja programa:

```
/> perm_mask  
> ls -l test_bar test_foo  
-rw-----1      dkrst0      Jan 16 16:23      test_bar  
-rw-rw-r--1      dkrst0      Jan 16 16:23      test_foo
```



Slika preuzeta iz [4].

Važni pojmovi

- Deskriptor ili opisnik datoteke je nenegativni cijeli broj koji se koristi za komunikaciju sa otvorenom datotekom i koji ju jedinstveno identificira.
- Standardni ulaz, izlaz, i izlaz za greške mogu se preusmjeravati.
- Postoji nekoliko standardnih funkcija za rad sa datotekama koje se koriste u programskom jeziku C: **open()**, **creat()**, **close()**, **lseek()**, **read()**, **write()**, i **umask()**.

Literatura

- [1] <https://pixabay.com/illustrations/ai-generated-id-card-identification-9667191/>
- [2] <https://pixabay.com/vectors/emergency-exit-green-white-309727/>
- [3] <https://pixabay.com/vectors/books-library-education-literature-42932/>
- [4] <https://pixabay.com/vectors/reminder-note-sticky-note-1922255/>
- [5]
- [6]
- [7]
- [8]